

column shows the test program size, the third column shows the test application time, while the last two rows show the sum of the test time of the corresponding cores. The fourth column gives the period of the test program during which the processor remains idle until the core completes its operation, i.e., the transmission of a frame. In the case that our methodology is used to test more than one core concurrently, we can take advantage of the idle periods of the processor in order to reduce the overall SoC test application time. For example, in our SoC benchmark we exploit the idle intervals in the SBST routines of one core in order to execute the SBST routines of the other. Especially during the idle intervals of the test routines for the Ethernet core the processor executes the test routines for the UART core achieving a test time reduction of more than 34% compared to the sequential test application case.

V. CONCLUSION

This paper describes a detailed study of the application of functional processor-based self-test on a broad class of communication peripherals of complex SoCs. The proposed methodology uses an embedded processor to apply a pure functional testing flow without any special test logic, unlike previous approaches for the testing of the nonprogrammable cores of the SoCs. Detailed experimental results on different implementations of two popular communication peripherals demonstrate the efficiency of the methodology.

REFERENCES

- [1] *IEEE Standard Testability Method for Embedded Core-Based Integrated Circuits*, 1500-2005, 2005 [Online]. Available: <http://www.ieee.org>
- [2] F. Corno, M. R. Sonza, G. Squillero, and M. Violante, "On the test of microprocessor IP cores," in *Proc. Des. Autom. Test Eur.*, 2001, pp. 209–213.
- [3] L. Chen, S. Ravi, A. Raghunathan, and S. Dey, "A scalable software-based self-test methodology for programmable processors," in *Proc. IEEE/ACM Des. Autom. Conf.*, 2003, pp. 548–553.
- [4] N. Kranitis, A. Paschalis, D. Gizopoulos, and G. Xenoulis, "Software-based self-testing of embedded processors," *IEEE Trans. Comput.*, vol. 54, no. 4, pp. 461–475, Apr. 2005.
- [5] C. Wen, L.-C. Wang, K.-T. Cheng, W.-T. Liu, and C.-C. Chen, "On a software-based self-test methodology and its application," in *Proc. IEEE VLSI Test Symp.*, 2005, pp. 107–113.
- [6] S. Gurumurthy, S. Vasudevan, and J. Abraham, "Automated mapping of pre-computed module-level test sequences to processor instructions," in *Proc. IEEE Int. Test Conf.*, 2005, paper 12.3, 10.1109/TEST.2005.1583987.
- [7] V. Singh, M. Inoue, K. Saluja, and H. Fujiwara, "Instruction-based self-testing of delay faults in pipelined processors," *IEEE Trans. Very Large Integr. (VLSI) Syst.*, vol. 14, no. 11, pp. 1203–1215, Nov. 2006.
- [8] M. Psarakis, D. Gizopoulos, M. Hatzimihail, A. Paschalis, A. Raghunathan, and S. Ravi, "Systematic software-based self-testing of pipelined processors," in *Proc. IEEE/ACM Des. Autom. Conf.*, 2006, pp. 393–398.
- [9] K. Jayaraman, V. M. Vedula, and J. A. Abraham, "Native mode functional self-test generation for systems-on-chip," in *Proc. Int. Symp. Quality Electron. Des.*, 2002, pp. 280–285.
- [10] S. Hwang and J. A. Abraham, "Reuse of addressable system bus for SoC testing," in *Proc. IEEE Int. ASIC/SoC Conf.*, 2001, pp. 215–219.
- [11] J.-R. Huang, M. K. Iyer, and K.-T. Cheng, "A self-test methodology for IP cores in bus-based programmable SoCs," in *Proc. IEEE VLSI Test Symp.*, 2001, pp. 198–203.
- [12] W.-C. Lai and K.-T. Cheng, "Instruction-Level DFT for testing processor and IP cores in system-on-a-chip," in *Proc. IEEE/ACM Des. Autom. Conf.*, 2001, pp. 59–64.
- [13] OpenCores, "OpenRISC 1200 Core, UART IP Core, Ethernet IP core, Wishbone Interconnection Architecture," (2007). [Online]. Available: www.opencores.org/projects/

Code Decompression Unit Design for VLIW Embedded Processors

Yuan Xie, Wayne Wolf, and Haris Lekatsas

Abstract—Code size "bloating" in embedded very long instruction word (VLIW) processors is a major concern for embedded systems since memory is one of the most restricted resources. In this paper, we describe a code compression algorithm based on arithmetic coding, discuss how to design decompression architecture, and illustrate the tradeoffs between compression ratio and decompression overhead, by using different probability models. Experimental results for a VLIW embedded processor TMS320C6x show that compression ratios between 67% and 80% can be achieved, depending on the probability models used. A precache decompression unit design is implemented in TSMC 0.25 μ m and a test chip is fabricated.

Index Terms—Code compression, embedded system.

I. INTRODUCTION

Memory is one of the most restricted resources in embedded systems. To reduce the code size for embedded processors, code compression [1] has been proposed as an effective approach and has been used by commercial products [2].

Very long instruction word (VLIW) embedded processors (such as TI's TMS320C6x [3]) are widely used for embedded systems. In order to explore instruction level parallelism (ILP), VLIW compilers use a lot of ambitious compiler techniques [4], which increase the code size. Therefore, compared to CISC or RISC processors, VLIW architectures have a "code size bloating" problem. It is, therefore, very important to reduce the code size for VLIW embedded processors.

In this paper, we describe a code compression algorithm (based on arithmetic coding) and the decompression unit design. Even though our previous work [5], [6] has demonstrated arithmetic coding-based code compression for RISC processor and proposed a variable-to-fixed-length code compression algorithm (V2FCC) for VLIW, the major contribution of this paper is how to design decompression architecture that allows for multibit decompression and illustrate the tradeoffs between compression ratio and decompression overhead by using different probability models.

II. CODE COMPRESSION ALGORITHM

In this section, we first give a brief introduction on the compression algorithm and the probability models and then demonstrate the compression result for an embedded VLIW processor TMS320C6x [3].

A. Binary Arithmetic Coding

Many previous code compression approaches used Huffman coding [1], [2]. In this section, we give a brief introduction on another code compression algorithm that uses *arithmetic coding* [7]. The most distinctive advantage for arithmetic coding is its separation of coding and modeling. It can be used together with any probability model that can

Manuscript received June 1, 2006; revised December 31, 2006.

Y. Xie is with the Computer Science and Engineering Department, the Pennsylvania State University, University Park, PA 16802 USA (e-mail: yuanxie@cse.psu.edu).

W. Wolf is with the Electrical Engineering Department, Princeton University, Princeton, NJ 08544 USA.

H. Lekatsas is with the NEC Laboratories, Princeton, NJ 08540 USA.

Digital Object Identifier 10.1109/TVLSI.2007.900755

TABLE I
EXAMPLE ENCODING MACHINE

State	Prob(MPB)	LPB	MPB
[0,8)	[0.8125,1)	000, [0,8)	-, [1,8)
	[0.625, 0.8125)	00, [0,8)	-, [2,8)
	[0.5, 0.625)	0, [0,8)	1, [0,8)
[1,8)	[0.7857,1)	001, [0,8)	-, [2,8)
	[0.6429, 0.7857)	0f, [0,8)	-, [3,8)
	[0.5, 0.6429)	0, [2,8)	1, [0,8)
[2,8)	[0.75, 1)	010, [0,8)	-, [3,8)
	[0.5, 0.75)	01, [0,8)	1, [0,8)
[3,8)	[0.7, 1)	011, [0,8)	1, [0,8)
	[0.5, 0.7)	ff, [0,8)	1, [2,8)

provide the probability distribution of the source. It provides great flexibility since the arithmetic coder does not need any change when the probability model is changed.

A reduced-precision arithmetic coding [7] can be implemented as a table lookup finite-state machine (FSM). For example, a simple four-state encoder is shown in Table I. Each state is represented by an interval; the starting state is represented by interval $[0, 8)$, while the other three possible states are intervals $[1, 8)$, $[2, 8)$, and $[3, 8)$. For each state, depending on the input bit and the probability of the input, we have different outputs and transitions. The probability ranges are shown in the second column. The input bit is either the most probable bit (MPB) or the least probable bit (LPB). For each input, the third and the fourth columns show the output bits and the next state (represented by an interval). The symbol “f” denotes the bits to follow procedure [7], which means that the output is not known yet and it depends on the next output bit. If the next output bit is a zero, then the “f” should be replaced by a one and *vice versa*.

B. Probability Models for Code Compression

In this section, we describe how to get the probability numbers that feed the FSM coder.

1) *Static Models*: A static model is a fixed model that fits all applications. The probability model is determined in advance from some sample applications. The simplest probability model is the one that assumes ones and zeros in the code have independent and identical distribution, and therefore, allocates a fixed probability for bit 1 and bit 0 irrespective of its position within instructions. For example, our experimental result indicate that for TMS320C6x, the average probability $P(0)$ is about 75%, and the deviation is quite small.

2) *Semiadaptive Finite-State Models*: The static model does not take into account the relationship and dependency among the input symbols. A more complicated statistical probability model is called Markov model, which consists of a number of states, where each state is connected to other states via two transitions. Starting from an initial state 0, the binary input stream is scanned. The left transition is taken when the input bit is 0 and right transition is taken when the input bit is 1. The number of transitions are recorded and used to calculate the probability for each transition. A Markov model can be described by two parameters: the *model depth* M and the *model width* N , which represent the number of layers and the number of Markov nodes per layer, respectively. Fig. 1 is an example of a $M \times N = 4 \times 4$ Markov model (note that not all the transition probabilities are shown in Fig. 1). The probability calculation is done as a preprocessing step, by going through the application code once and traversing the paths according to the bits encountered in the program.

C. Compressing VLIW Instructions

In this section, we present the compression results for TMS320C6x VLIW processor [3]. Most of the benchmarks are provided by Texas Instruments and compiled using TI's *Code Composer Studio IDE*.

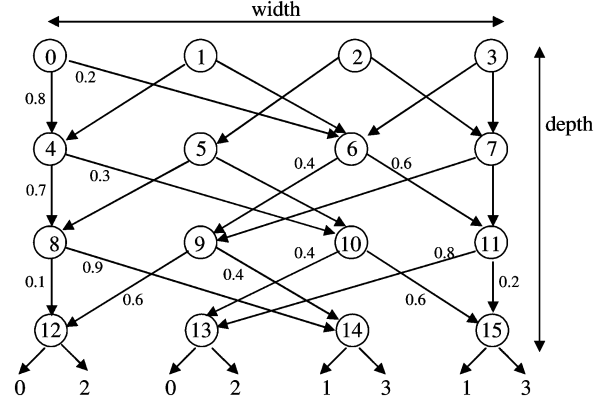


Fig. 1. Example 4×4 Markov model.

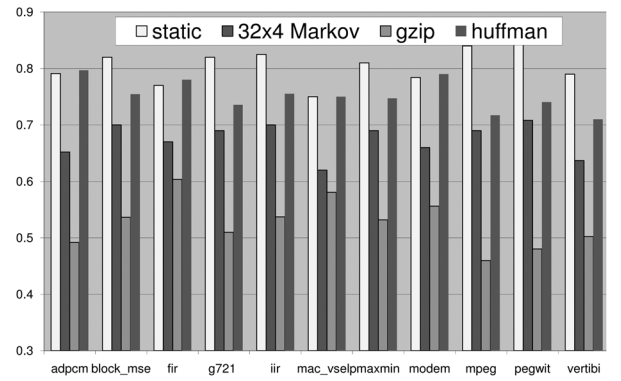


Fig. 2. Compression ratio for TMS320C6x and the comparison with GZIP coding and Huffman coding.

We use a four-state FSM (see Table I) for all the experiments. In Fig. 2, the first bar and the second bar show the compression ratio for TMS320C6x by using a static model and a 32×4 Markov model, respectively. Experiments show that we can achieve an average compression ratio 80% and 67%, respectively. We compare our algorithm with two well-known compression algorithms: the third bar and the fourth bar show the compression ratio achieved by using GZIP (which uses LZ77 [8] coding algorithm) and a Huffman encoder, respectively. It shows that our results are much worse than the GZIP approach. However, GZIP compress the whole program without dividing the file into blocks and there for not suitable for code decompression. The Markov model-based approach outperforms Huffman coding, since the Huffman encoder does not take into account the dependencies between bytes, hence its performance is limited. The static model-based approach sometimes is better than the Huffman approach and sometimes is worse, but on average is about 3% worse.

III. DECOMPRESSION ARCHITECTURE DESIGNS

Using the encoder FSM and the probability models in Section II, one can easily construct a decoder FSM and decompress the compressed code. However, the main drawback of decompression is that it works on a bit-by-bit basis and results in slow decoding. In this section, we discuss how to design a multibit decompression unit.

A. Multibit Decompression Algorithm for Static Models

The first step is to construct the decoder FSM and check if the FSM can be simplified, by removing those states and the transitions that are never reached. For example, for TMS320C6x, a static model gives $\text{Probability}(1) = 0.25$ and $\text{Probability}(0) = 0.75$. The decoder can

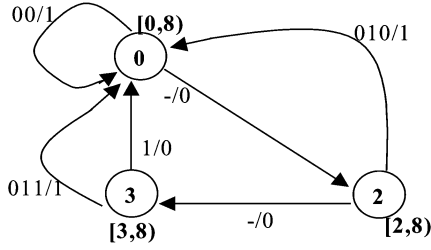


Fig. 3. Decoder FSM.

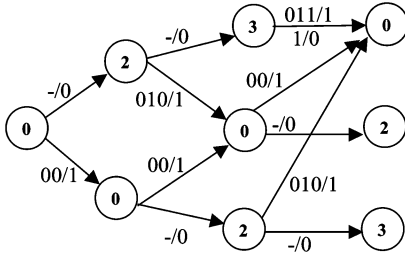


Fig. 4. Decoder tree with depth = 3.

TABLE II
EXAMPLE MULTIBIT DECODER FOR STATIC MODEL

Current State	Encoded bits	Decoded bits	Shift	Next state
0	1	000	1	0
	011	001	3	0
	010	01	3	0
	00	1	1	0

be simplified to be a three-state FSM machine as shown in Fig. 3, by removing state [1, 8] and other transitions that are never reached and replacing LPB with 1 and MPB with 0.

The next step is to build a decoder tree by following all possible paths starting from state 0 (for simplicity, we denote state $[i, 8]$ to be state i in the following discussion), since we always start from state 0 when we begin to compress/decompress a block. Fig. 4 shows a decoder tree with depth = 3 for the FSM in Fig. 3. Based on the decoder tree, we can design a multibit decoder. For example, it is easy to show that if the encoded bits are 011, the decoder would produce output 001. Using a subset of the decoder tree, where all ending nodes are state 0, we can construct a multiple-bit decoder FSM with four transitions, as shown in Table II. Note that each transition comes back to state 0. Therefore, the decoder is a one-state FSM with four transitions back to itself. We can expand the encoder tree further to get more bits to be decoded at each clock cycle. Though it is not necessary, we can try to ensure that the ending node in the subset of the decoder tree are all state 0, such that the resulting decoder is a one state FSM with more transitions. For example, a one-state FSM with 21 transitions can decode up to six bits per each cycle.

To design such a multiple-bit decoder, one must pay attention to the following two issues.

- 1) *Any compressed binary stream should be parsable by the decoder.* For example, using the multibit decoder table in Table III, input bits 001 cannot find any matched transition in the table and cannot be decoded (even though it can be decoded to be 1000 by using Table II). In order to make sure that all binary streams can be decoded, when designing the decoder table, one must choose a subset of the decoder tree such that all transitions from the nodes in the subset are ended within

TABLE III
A MULTIBIT DECODER ARE NOT PARSABLE

Current State	Encoded bits	Decoded bits	Shift	Next state
0	1	000	1	0
	011	001	3	0
	010	01	3	0
	0000	11	4	0

TABLE IV
INVALID DECODER THAT CANNOT DISTINGUISH ALL BINARY STREAMS

Current State	Encoded bits	Decoded bits	Shift	Next state
0	1	000	1	0
	011	001	3	0
	01000	011	5	0
	010	010	3	2
	000000	111	6	0
	0000	110	4	2
	00010	101	5	0
	00	100	2	3

the subset, except for the ending nodes. Such a subset tree is defined to be *complete*.

- 2) *Any compressed binary stream must be distinguishable.* For example, if we take the third level nodes (state 0, state 2, and state 3) as the ending nodes, the decoder tree is complete. The result decoder table is shown in Table IV. If the input bits are 000000, there are several ways to parse it: the first two bits 00 are decoded as 100 and the next state is 3; or the first four bits 0000 are decoded as 110 and the next state is 2; or the whole six bits are decoded as 111 and the next state is 0. The decoder cannot differentiate these situations. To avoid this problem, the subset decoder tree must satisfy the following requirements.

- 1) Any ending nodes in the subset tree can not have any in-edge with empty input (i.e., the “-” on the transition edge).
- 2) No transition between ending nodes (otherwise, the decoder cannot distinguish which ending node is the next state).

B. Multibit Decompression Algorithm for Markov Models

For adaptive Markov model, the multibit decompression scheme is more complicated than that for static model. Fig. 5 shows a decoder tree that uses Markov model. The decoding starts from Markov state 0 and FSM state $[0, 8]$; the left edge is for input bit 0 and the right edge is for input bit 1, with probability 0.8 and 0.2, respectively. When the encoded bits are 00, the decoded bit is the LPB (denoted as L in Fig. 5); the next decoder state is $[0, 8]$, and the encoded bits are shifted out two bits; otherwise, the output bit is MPB (denoted as M in Fig. 5), the next decoder state is $[2, 8]$ and no shift for encoded bits. Following the same manner, a decoder tree is constructed and each node in the decoder tree is actually a combination of one encoder state and one Markov state.

After constructing such a decoder tree, a multibit decoder table can be extracted, following the requirement and procedure that are discussed in the previous subsection. For example, the selected ending nodes are those with shadow ones. The corresponding decoder entry in the decoder table is shown in Table V. The entry has six matches (transitions). For example, if the encoded bit is “1,” which matches the first row in this entry and can be decoded as MM(01). We shift one bit of the encoded bit stream and the next entry state is $\{[0, 8], 0\}$.

Table V only shows the entry for the state $\{[8, 0], 0\}$. Each entry state in the decoder table is a combination of one encoder state and one Markov state. The upper bound of the number of the entries would be the number of the encoder states multiplied by the number of the

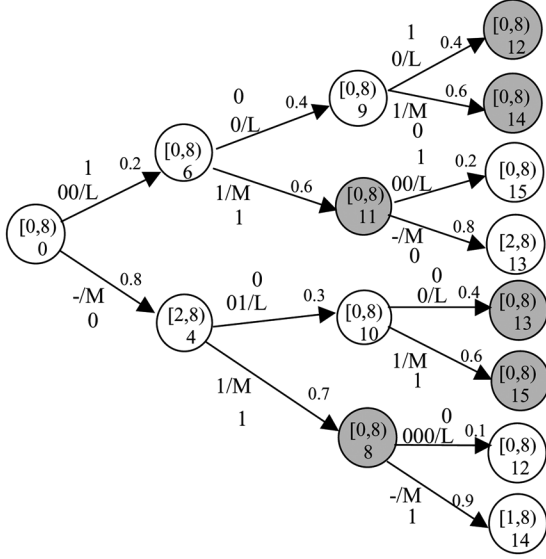


Fig. 5. Decoder tree which is a combination of decoder table and Markov model.

TABLE V
EXAMPLE DECODER FOR STATE $\{[8, 0], 0\}$

Current State	Encoded bits	Decoded bits	Shift	Next state
[0,8],0	1	MM(01)	1	[0,8],8
	010	MLL(000)	3	[0,8],13
	011	MLM(001)	3	[0,8],15
	001	LM(11)	3	[0,8], 11
	0000	LLL(101)	4	[0,8],12
	0001	LLM(100)	4	[0,8],14

Markov states. In order to keep the decoder table small we have to keep the number of states both in FSM decoder and in Markov model to be small. In practice, a four-state FSM machine and a 256-state Markov model can achieve a good compression ratio and give an upper bound of 1-K entries in the decoder table. For each entry, depending on the number of bits to be compared, the number of matches varies. From Fig. 5, one may observe that by looking ahead more symbols, the size of each decoding table entry grows. However, the growth is not exponential, as we have shown that some nodes in the decoder cannot be valid ending nodes, and therefore, cannot be a valid entry in the decoder table. Therefore, the total number of matches in the decoder table does not increase exponentially with the depth of the expanded state graph, and the number of entries is much fewer than the upper bound (number of encoder states multiplied by number of Markov states). In practice, many combinations of the FSM states and Markov states do not appear in the decoder table. Our experiments show that the typical total entries of the decoder table did not exceed 500 (for a four-state FSM machine and a 256-state Markov model), while the maximum number of matches for each entry depends on how many bits can be decoded simultaneously, which varies from 1 to 16.

C. Decompression Architecture Implementation

In this section, we present the implementation of the decompression unit design.

1) *Implementation of Multibit Decompression Scheme for Markov Models:* We first discuss the implementation of a multibit decompression scheme using a Markov model, which is more complicated than a static model. Based on our multibit decompression scheme, the decoder table has many entries similar to the one shown in Table V. Each

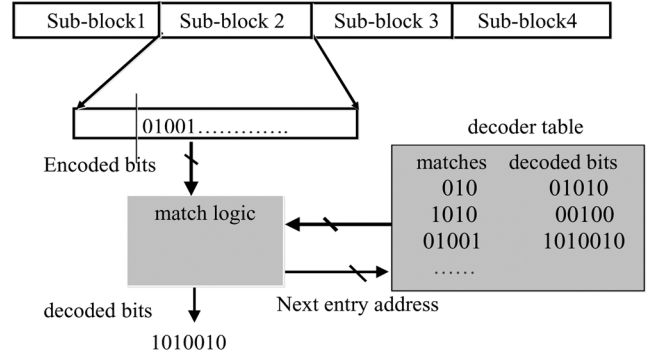


Fig. 6. Multiple bit decoding process.

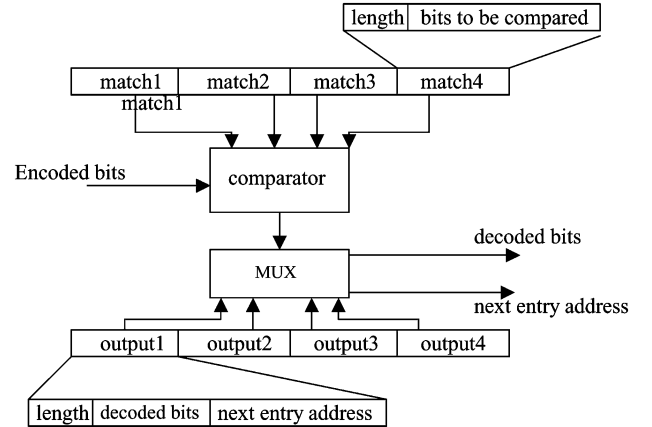


Fig. 7. Match logic does comparison and multiplexing.

entry is a combination of the encoder FSM state and the Markov model state. Each entry has a number of matches. These matches are compared against the encoded bits and the corresponding decoded bits are generated, so that multiple bits are decoded per cycle. This table is generated during the encoding phase and is application specific, since each application has its distinctive Markov model.

The decoding procedure is actually a comparison of the encoded bits stream with the decoder table matches. Fig. 6 shows the decoding process, where the match logic compares the encoded bits with the matches in the decoder table, selects the output, and produces the next entry address. Since the decoder table is a variable-to-variable length mapping, the match logic has to know how many bits are to be compared, and, if there is a match, how many bits are to be sent to output and what is the next entry (i.e., the last column in Table V). Fig. 7 shows the behavior of the match logic. The matches and outputs are from the decoder table. Each match indicates how many bits are to be compared with the encoded bits and what are the bits to be compared. The outputs include decoded bits and the next entry address. The comparator controls the multiplexor to choose the correct output and send the next entry address to the decoder table.

A RAM or a ROM can be used to store the decoder table. The advantage of using RAM to implement the decoder table is that when the application changes, we can update the decoder table. The advantage of using ROM is that the area is smaller than a RAM. If the application running on the embedded system is fixed, we can use a ROM to implement the decoder to save area.

Assume that the maximum matches for each entry is N . In order to compare the N matches simultaneously, the design of the decoder table is similar to an N -way cache design. The decoder core is shown in Fig. 8. The input buffer accepts input from the instruction memory.

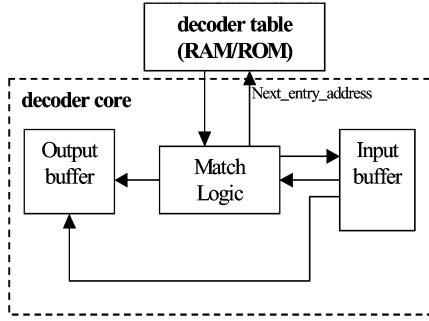


Fig. 8. Decoder core.

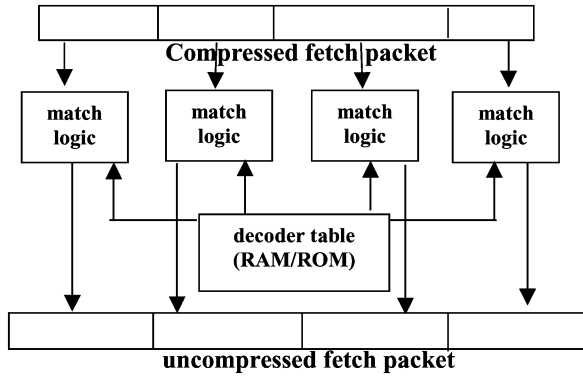


Fig. 9. Example parallel multiple bits decompression architecture for TMS320C6x.

If the block is uncompressed, it is sent to the output buffer directly (during compression, if the compressed block size is larger than the original size, we keep that block uncompressed). It also contains the shifter that is used to shift the input bits. The output buffer accepts uncompressed instruction from the input buffer directly or from the decoder. The match logic consists of N comparators and a N -to-1 multiplexor. The inputs to the match logic are from the decoder table and the input buffer. The multiplexor selects the correct match from the comparators and sends the output bits to the output buffer. It also tells the input buffer how many bits have been matched and therefore must be shifted out. The next entry address is sent to the decoder table, which are stored in RAM or ROM. Fig. 6 only shows decoding for one sub-block. Fig. 9 illustrates a decoder architecture for TMS320C6x, where the fetch packet is divided to be four subblocks such that each subblock can be decompressed simultaneously with other subblocks.

Based on the architecture previously described, we implemented and fabricated a test chip via MOSIS. The test chip is a four-way parallel decoder as shown in Fig. 9. The design assumes that the maximum number of entries is 512 (corresponds to the upper bound for an four-state encoder with a 32×4 Markov model), and the maximum matches for each entry is four. We modeled the architecture in VHDL, and then used the Design Compiler by Synopsys to synthesize the circuit into gate level netlists, using a TSMC 0.25- μm standard cell library. The decoder table is stored in RAMs, such that the decoder is programmable. The gate level netlist is imported into the Silicon Ensemble by Cadence to do placement and routing. The exported GDSII is imported into the layout editor Virtuoso by Cadence and verification is also done by Cadence's tools. The test chip was sent to MOSIS for fabrication. The chip size is $3.4 \text{ mm} \times 3.4 \text{ mm}$, in which the decoder table RAMs occupy about 95% of the chip area. Therefore, using a multiport RAM or ROM to store the decoder table that can be shared by all decoder cores is critical to design an area-efficient decoder.

The average decompression speed is about 11 bits per clock cycle at 45 MHz, corresponding to 465 Mbits/s. Since the size of a fetch packet in TMS320C6x is 256 bits, it takes more than 23 cycles to decompress a fetch packet. Note that the decompressor is placed between the cache and the memory. Therefore, decompression happens only when there is a cache miss, and the decompression overhead can be factorized into the cache miss penalty.

2) Implementation of Multibit Decompression for Static Models:

The structure of the decoder table for static model is similar to the one used for the Markov model, but the total number of the entries in the decoder is greatly reduced. For instance, considering a four-state encoder combined with a 32×8 Markov model, the upper bound of the number of entries for the decoder table is 1024 ($4 \times 32 \times 8$). On the contrary, a four-state encoder combined with a static model has only four entries in the decoder table. In practice, not all combinations occur. For instance, using the same benchmarks for Fig. 3, a decoder table for a 32×8 Markov model has about 500 states and an average of 1500 matches when allowing up to three bits to be decoded per cycle. On the contrary, for a static model, it is not necessary to store such a small amount of decoder table in a ROM since it is straightforward to design it as a FSM.

Using the same technology (TSMC 0.25 μm), we implemented a decompression unit that uses static model. A one-state decoder with 21 transitions can decode up to six bits per cycle, and its implementation is only 0.01 mm^2 , with around 200 logic cells. The average decompression speed is 4.9 bits per cycle, thus, it takes average 52 cycles to decompress one fetch packet for TMS320C6x. Therefore, by using a static simple model, the decompression unit is much smaller than the one that uses a Markov model. Furthermore, another significant advantage of using a static model is that the decompression unit is not application specific and does not need any change when the application running on the system changes.

3) Comparison With IBM Codepack: We compare our design against IBM's decompression core CodePack [2] (which uses Huffman coding and is only for RISC processor). The CodePack core is also implemented in 0.25- μm CMOS technology and consists of 25484 logic cells, occupying 1- mm^2 silicon area. The size of our decoder core with Markov model is only about 10% of their decoder core. IBM reports that their decoder table ROM for two 512-entry tables is 10814 cells (however, the silicon area for the decoder table ROM is not mentioned). If we had implemented the decoder table in ROM rather than in RAM, we could have achieved a smaller silicon area for the core. Note that our decoder core using static model is much smaller (0.01 mm^2), and the compression ratio is comparable to the Huffman approach (see Section II-C).

IV. CONCLUSION

In this paper, we present a code compression algorithm based on reduced arithmetic coding using two different probability models. We demonstrate how to design decompression architecture and illustrate the tradeoffs between compression ratio and decompression overhead with different probability models. Since the decompression unit is placed between the main memory and the cache, decompression happens only on a cache miss, and the decompression overhead can be factorized into the cache miss penalty.

REFERENCES

- [1] A. Wolfe and A. Chanin, "Executing compressed programs on an embedded RISC architecture," in *Proc. Int. Symp. Microarch.*, 1992, pp. 81–91.
- [2] T. M. Kemp, R. K. Montoye, J. D. Harper, J. D. Palmer, and D. J. Auerbach, "A decompression core for powerPC," *IBM J. Res. Develop.*, vol. 42, no. 6, pp. 807–812, Nov. 1998.

- [3] Texas Instruments, Austin, "TMS320C62xx CPU and instruction set: Reference guide," 1997.
- [4] J. Fisher, P. Faraboschi, and C. Young, *Embedded Computing: A VLIW Approach to Architecture, Compiler, and Tools*. San Mateo, CA: Morgan Kaufmann, 2005.
- [5] H. Lekatsas and W. Wolf, "SAMC: A code compression algorithm for embedded processors," *IEEE Trans. Comput.-Aided Des. Integr. Circuits Syst.*, vol. 18, no. 12, pp. 1689–1701, Dec. 1999.
- [6] Y. Xie, W. Wolf, and H. Lekatsas, "Code compression for embedded vliw processors using variable-to-fixed coding," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 14, no. 5, pp. 525–536, May 2006.
- [7] P. G. Howard and J. S. Vitter, "Parallel lossless image compression using Huffman and arithmetic coding," *Inf. Process. Lett.*, vol. 59, pp. 65–73, 1996.
- [8] J. Ziv and A. Lempel, "A universal algorithm for sequential data compression," *IEEE Trans. Inf. Theory*, vol. 23, no. 3, pp. 337–343, May 1977.